

Kiến Trúc Micro Frontend — Lishop

Ngày tạo: 27/06/2026

1. Tổng Quan

Lishop sử dụng kiến trúc **Micro Frontend** với monorepo quản lý bởi **pnpm workspaces + Turborepo**. Hệ thống gồm **10 Next.js 15 apps** (MFEs), mỗi app chạy trên một port riêng và hoàn toàn độc lập.

Thành phần	Công nghệ
Package Manager	pnpm v9 với workspaces
Build Orchestration	Turborepo v2
Framework	Next.js 15 (App Router)
Language	TypeScript 5.5
Styling	Tailwind CSS v4
UI Library	Custom @lishop/ui (Radix primitives + CVA)
Micro-FE Pattern	URL-Based Navigation (full-page redirects)

2. Danh Sách Apps

2.1 Shell (Host) — Port 3010

App chính đóng vai trò là entry point của toàn bộ hệ thống.

Chức năng:

- Trang chủ (landing page) với hero section, featured products, flash sales
- Global layout: Header, Footer, AnnouncementBar
- Auth state management (Zustand store)
- AI Chat widget (hiển thị trên mọi trang)
- Global notification badges

2.2 Các Micro Frontend Apps

App	Port	Routes	Chức năng
mfe-auth	3001	/login, /register, /forgot-password, /reset-password, /verify-email	Xác thực người dùng
mfe-catalog	3002	/products, /products/[slug], /shops/[slug]	Duyệt sản phẩm, AI concierge
mfe-cart	3003	/cart	Giỏ hàng
mfe-checkout	3004	/checkout	Thanh toán (3 bước)
mfe-orders	3005	/orders, /orders/[id]	Đơn hàng, trả hàng

mfe-profile	3006	/profile, /addresses, /wallet, /wishlist, /support	Hồ sơ người dùng
mfe-promotions	3007	/promotions	Coupons & flash sales
mfe-notifications	3008	/notifications	Trung tâm thông báo
mfe-admin	3009	/admin	Dashboard quản trị
mfe-seller	3011	Seller dashboard, inventory, analytics, chat	Không gian người bán

3. Shared Packages

3.1 @lishop/event-bus — Giao Tiếp Liên App

Sử dụng **BroadcastChannel API** để giao tiếp real-time giữa các tab/window.

```
const CHANNEL_NAME = 'lishop-events';

class LishopEventBus {
  private channel: BroadcastChannel;

  constructor() {
    this.channel = new BroadcastChannel(CHANNEL_NAME);
    this.channel.onmessage = ({ data }) => { /* dispatch listeners */ };
  }

  on(event, handler) { /* register */ }
  off(event, handler) { /* unregister */ }
  emit(event, payload) { /* broadcast */ }
}
```

Các sự kiện:

Event	Payload	Trigger
AUTH_LOGIN	{ userId, role }	Đăng nhập thành công
AUTH_LOGOUT	undefined	Đăng xuất
CART_UPDATED	{ itemCount }	Thêm/xóa item trong giỏ
CART_CLEARED	undefined	Hoàn tất thanh toán
ORDER_PLACED	{ orderId, orderNumber }	Đặt hàng thành công
NOTIFICATION_RECEIVED	{ notificationId }	Nhận thông báo real-time
NOTIFICATION_COUNT_UPDATED	{ count }	Số lượng thông báo thay đổi
PROFILE_UPDATED	{ userId, firstName?, lastName?, avatarUrl? }	Cập nhật hồ sơ

3.2 @lishop/shared — Tiện Ích Dùng Chung

API Client: `createApiFetch(url, authUrl)` — factory function tạo API client với auto-refresh khi 401, timeout, `credentials: 'include'` .

Auth: `hasSessionCookie()` — kiểm tra cookie `lishop_session=1` .

Hooks:

- `useAuthSync()` — đồng bộ trạng thái auth qua BroadcastChannel
- `useNotificationStream()` — WebSocket notifications (Socket.IO)
- `useRealtime()` — Generic Socket.IO hook
- `useOrderRealtime()` — Real-time cập nhật đơn hàng
- `useProductRealtime()` — Real-time tồn kho & flash sale
- `useShopChat()` / `useTicketChat()` — Chat real-time
- `useAdminFeed()` — Admin feed real-time

Formatters: `formatVND()` , `formatUSD()` , `formatDate()`

Utils: `cn()` (clsx + tailwind-merge)

Catalog: `productListUrl()` , `getProductListFiltersFromSearchParams()`

3.3 @lishop/ui — Component Library

Radix UI primitives wrapped với **class-variance-authority (CVA)**:

Accordion, Alert, AlertDialog, Avatar, Badge, Button, Card, Checkbox, Dialog, DropdownMenu, Input, Label, Popover, Progress, RadioGroup, ScrollArea, Select, Separator, Sheet, Skeleton, Switch, Table, Tabs, Textarea, Toast (sonner), Tooltip

3.4 @lishop/contracts — Zod Validation Schemas

Schemas: LoginSchema, RegisterSchema, ForgotPasswordSchema, ResetPasswordSchema, ProductSchema, CartSchema, OrderSchema, PaymentSchema, ShippingSchema, ReviewSchema, PromotionSchema, NotificationSchema

Enums: UserRole, OrderStatus, PaymentMethod, Currency

3.5 @lishop/config — Shared Config

- ESLint: Next.js + TypeScript + Prettier
- Tailwind: Base theme (colors, border radius)
- TypeScript: Base tsconfig (ES2022, bundler module resolution, strict mode)

4. Các MFEs Kết Nối Với Nhau Như Thế Nào?

4.1 URL-Based Navigation (Phương thức chính)

Đây **không phải** Module Federation kiểu load remote components runtime. Mỗi MFE là một Next.js app riêng biệt, chuyển trang bằng **full-page navigation**.

Shell header chứa các link cứng đến từng MFE:

```
const MFE = {
  auth: process.env['NEXT_PUBLIC_MFE_AUTH_URL'] ?? 'http://localhost:3001',
  catalog: process.env['NEXT_PUBLIC_MFE_CATALOG_URL'] ?? 'http://localhost:3002',
  cart: process.env['NEXT_PUBLIC_MFE_CART_URL'] ?? 'http://localhost:3003',
  orders: process.env['NEXT_PUBLIC_MFE_ORDERS_URL'] ?? 'http://localhost:3005',
  profile: process.env['NEXT_PUBLIC_MFE_PROFILE_URL'] ?? 'http://localhost:3006',
  promotions: process.env['NEXT_PUBLIC_MFE_PROMOTIONS_URL'] ?? 'http://localhost:3007',
  notifications: process.env['NEXT_PUBLIC_MFE_NOTIFICATIONS_URL'] ?? 'http://localhost:3008',
  admin: process.env['NEXT_PUBLIC_MFE_ADMIN_URL'] ?? 'http://localhost:3009',
  seller: process.env['NEXT_PUBLIC_MFE_SELLER_URL'] ?? 'http://localhost:3011',
};
```

Khi click link → trình duyệt tải trang mới hoàn toàn từ MFE khác.

4.2 BroadcastChannel (Event Bus)

Dùng để gửi sự kiện real-time giữa các tab/window đang mở. Chi tiết các sự kiện ở mục 3.1.

4.3 localStorage

Dùng để đồng bộ state đơn giản giữa các app:

- `lishop_cart_count` — số lượng items trong giỏ hàng
- `lishop_notification_count` — số thông báo chưa đọc

Các app lắng nghe sự kiện `storage` của window để cập nhật badge tương ứng.

4.4 Cookies (Auth Session)

Backend set các `httpOnly` cookies:

- `lishop_at` — access token (`httpOnly`, không đọc được từ JS)
- `lishop_session=1` — JS-readable indicator

FE chỉ check `hasSessionCookie()` để biết user đã login chưa.

5. Các MFEs Kết Nối Với Backend Như Thế Nào?

5.1 API Client Chung

Tất cả MFE đều gọi chung backend API qua `createApiFetch` từ `@lishop/shared`:

```
function createApiFetch(apiUrl: string, authUrl: string) {
  return async function apiFetch<T>(path, init = {}): Promise<T> {
    // 1. Request với credentials: 'include' (tự động gửi cookies)
    let res = await fetch(`${apiUrl}${path}`, {
      credentials: 'include',
      ...init
    });

    // 2. Nếu 401 → tự động gọi /auth/refresh
    if (res.status === 401) {
      const refreshRes = await fetch(`${apiUrl}/auth/refresh`, {
```

```

        method: 'POST',
        credentials: 'include',
    });

    if (!refreshRes.ok) {
        // Redirect về trang login nếu refresh thất bại
        window.location.href = `${authUrl}/login`;
        throw new Error('Session expired');
    }

    res = await doRequest(path, init); // Retry request gốc
}

// 3. Parse & unwrap { data: ... } envelope
const json = await res.json();
return (json.data ?? json) as T;
};
}

```

5.2 Cấu Hình Mỗi App

Mỗi MFE tự tạo instance API riêng:

```

const apiFetch = createApiFetch(
  process.env['NEXT_PUBLIC_API_URL'] ?? 'http://localhost:4000',
  process.env['NEXT_PUBLIC_MFE_AUTH_URL'] ?? 'http://localhost:3001',
);

```

5.3 Real-time (Socket.IO + SSE Fallback)

Dùng Socket.IO kết nối đến backend:

- `useNotificationStream` — nhận notification real-time
- `useOrderRealtime` — lắng nghe `order:{id}` room
- `useProductRealtime` — lắng nghe `product:{id}:stock`, `flashsale:{id}`
- `useShopChat` / `useTicketChat` — real-time chat cho seller

Nếu WebSocket không kết nối được, tự động fallback sang SSE.

6. Auth Flow Chi Tiết

1. Shell load
2. AuthInitializer (Providers.tsx) chạy
3. Gửi GET /auth/me (cookies tự động gửi qua credentials: 'include')
4. Nếu 401 → tự động gọi POST /auth/refresh
5. Nếu cả hai đều fail → `clearAuth()` (user là anonymous)
6. Login thành công (từ mfe-auth hoặc shell):
 - a. POST /auth/login → backend set httpOnly cookies
 - b. Fetch /auth/me → lấy user profile
 - c. `setAuth(user, accessToken)` trong Zustand store
 - d. `eventBus.emit(AUTH_LOGIN)` → đồng bộ qua các tab

7. Auth sync cross-MFE:
 - useAuthSync() hook lắng nghe AUTH_LOGIN / AUTH_LOGOUT
 - Khi nhận AUTH_LOGOUT → clearAuth() toàn bộ
8. Protected pages redirect về {auth}/login nếu chưa login

Auth Store (Zustand):

```
interface AuthState {  
  user: AuthUser | null;  
  accessToken: string | null;  
  setAuth: (user: AuthUser, accessToken: string) => void;  
  clearAuth: () => void;  
}
```

7. Luồng Dữ Liệu Mẫu

7.1 Request Trang Thông Thường

Browser → Shell (localhost:3010)

- Shell render layout + header
- User click "Sản phẩm"
- Full navigation tới {catalog}/products?q=keyword
- mfe-catalog render độc lập
- Gọi API tới localhost:4000/products
- Dùng QueryClient riêng (TanStack Query)

7.2 Thêm Item Vào Giỏ Hàng

1. User ở mfe-catalog click "Thêm vào giỏ"
2. mfe-catalog gọi PUT /cart/items tới backend
3. Backend trả về itemCount mới
4. Cart app lưu localStorage.setItem('lishop_cart_count', count)
5. eventBus.emit(CART_UPDATED, { itemCount })
6. Shell header lắng nghe:
 - storage event → cập nhật badge
 - BroadcastChannel → cập nhật badge
7. Các tab khác cũng cập nhật badge tương ứng

7.3 Real-time Notification

1. Backend emit WebSocket event 'notification'
2. useNotificationStream() hook nhận event
3. Hiển thị toast (sonner)
4. Tăng localStorage notification count
5. eventBus.emit(NOTIFICATION_COUNT_UPDATED)
6. Tất cả MFE có badge cập nhật số lượng

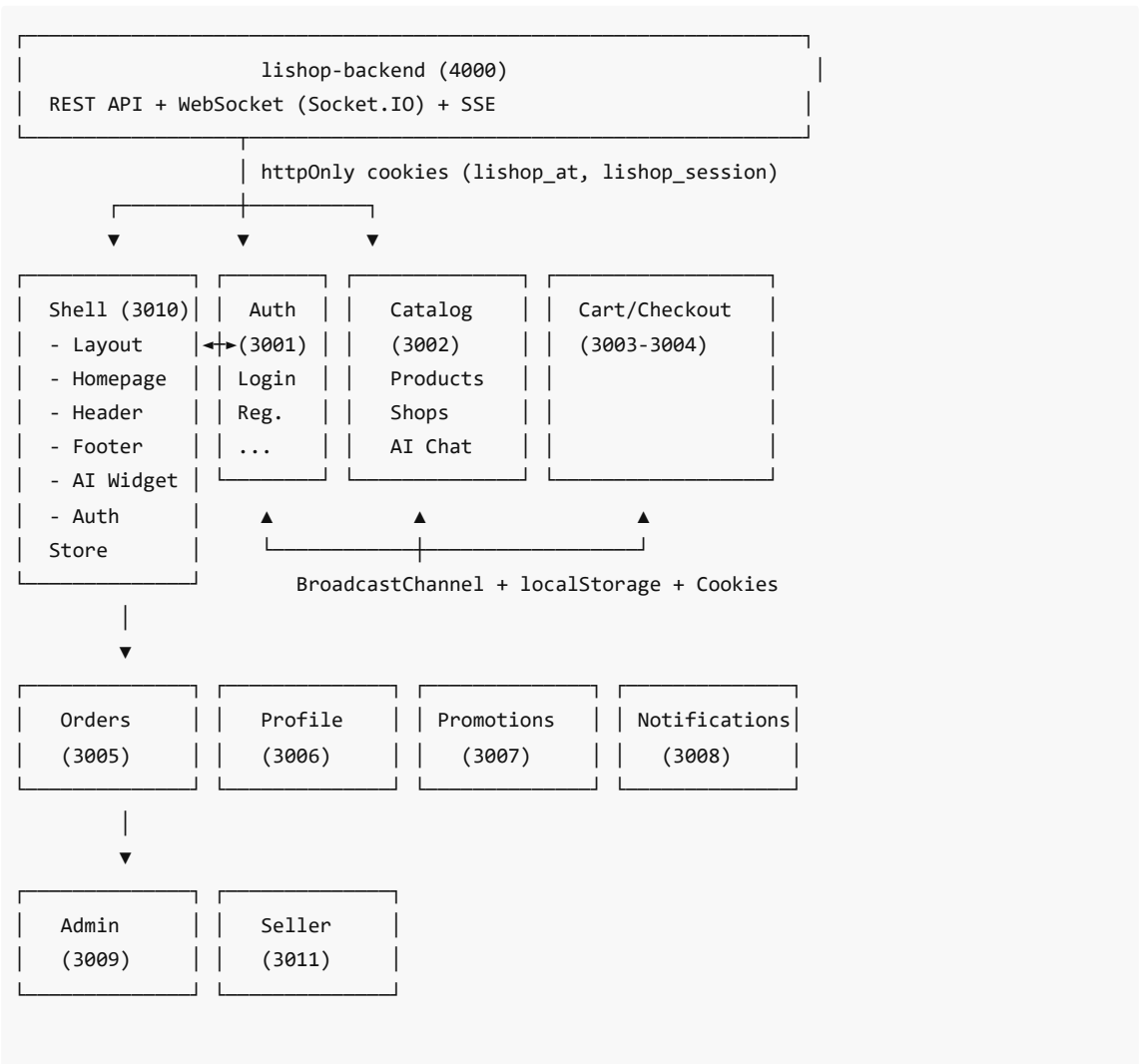
8. Environment Variables

File `.env` ở root workspace:

```
NEXT_PUBLIC_API_URL=http://localhost:4000
NEXT_PUBLIC_SHELL_URL=http://localhost:3010
NEXT_PUBLIC_MFE_AUTH_URL=http://localhost:3001
NEXT_PUBLIC_MFE_CATALOG_URL=http://localhost:3002
NEXT_PUBLIC_MFE_CART_URL=http://localhost:3003
NEXT_PUBLIC_MFE_CHECKOUT_URL=http://localhost:3004
NEXT_PUBLIC_MFE_ORDERS_URL=http://localhost:3005
NEXT_PUBLIC_MFE_PROFILE_URL=http://localhost:3006
NEXT_PUBLIC_MFE_PROMOTIONS_URL=http://localhost:3007
NEXT_PUBLIC_MFE_NOTIFICATIONS_URL=http://localhost:3008
NEXT_PUBLIC_MFE_ADMIN_URL=http://localhost:3009
```

Turborepo config có `NEXT_PUBLIC_*` trong `globalEnv` , giúp tất cả apps đều truy cập được các biến môi trường.

9. Sơ Đồ Kiến Trúc



Shared Packages (@lishop/):

ui	shared	event-bus	contracts	config
(Radix)	(utils,	(Broadcast	(Zod	(ESLint,
	hooks,	Channel)	schemas)	Tailwind,
	API)			TS config

10. Các Quyết Định Kiến Trúc Quan Trọng

Quyết định	Lựa chọn	Lý do
Federation pattern	URL-Based Navigation	Đơn giản, deploy độc lập, không cần runtime module loading
Auth mechanism	httpOnly Cookies	An toàn hơn client tokens, chống XSS
State management	Zustand (global) + React Query (local)	Nhẹ, đủ dùng cho auth state; mỗi MFE tự quản lý data riêng
Cross-tab communication	BroadcastChannel + localStorage	Không cần backend service cho việc đồng bộ đơn giản
Shared types	Zod contracts	Validate cả FE và BE, type-safe
Real-time	Socket.IO + SSE fallback	WebSocket ưu tiên, fallback khi không kết nối được
Styling	Tailwind CSS v4	Utility-first, dễ maintain
UI primitives	Radix UI + CVA	Accessible, customizable

11. Deployment Isolation

Mỗi MFE có thể được deploy độc lập nhờ:

- Chạy trên port riêng (3010-3011)
- Codebase riêng trong `apps/` directory
- Package.json riêng với scripts riêng
- Không phụ thuộc runtime vào app khác

Tuy nhiên, vì dùng monorepo, các apps chia sẻ:

- node_modules (pnpm workspace)
- ESLint, TypeScript, Tailwind config
- Shared packages (`@lishop/*`)
- Environment variables (Turborepo globalEnv)

12. Danh Sách Công Nghệ

Công nghệ	Version	Mục đích
-----------	---------	----------

Next.js	15	Framework chính cho tất cả apps
TypeScript	5.5	Ngôn ngữ lập trình
pnpm	9	Package manager
Turborepo	2	Build orchestration
Tailwind CSS	4	Styling
Radix UI	Latest	UI primitives
class-variance-authority	Latest	Component variants
TanStack Query	Latest	Data fetching & caching
Zustand	Latest	Global state management
Socket.IO	Latest	Real-time communication
zod	Latest	Schema validation
sonner	Latest	Toast notifications
clsx + tailwind-merge	Latest	Class name utilities
BroadcastChannel	Native API	Cross-tab communication